TUGAS PRAKTIK PEMROGRAMAN BERBASIS OBJEK **TUGAS PERTEMUAN 10**

Nama : M. Davana Yudista Firdausy

Nrp : 3125522002

Kelas : A D3 TI-SM

1. KEGIATAN PRAKTIKUM

- **Langkah 1 — Analisis UML:** Mengidentifikasi antarmuka `PelacakGPS`, kelas abstrak `Kendaraan`, kelas konkret `MobilReguler` dan `MobilVIP`, serta pengelola agregasi `PerusahaanRental` beserta atribut, tipe data, dan derajat visibilitasnya.
- **Langkah 2 — Pembuatan Struktur Class:** Menyusun berkas kode ke dalam paket modular di bawah `SourceCode/`, meliputi direktori `model/`, `server/`, dan antarmuka web di direktori `web/`.
- **Langkah 3 — Implementasi Attribute & Enkapsulasi:** Menjaga seluruh atribut berstatus `private` atau `protected` dengan method accessor/mutator yang terkontrol.
- **Langkah 4 — Implementasi Constructor & Method:** Mengimplementasikan konstruktor berparameter, method `hitungBiayaRental()`, serta pemutakhiran telemetri `updatePosisi()`.
- **Langkah 5 — Implementasi Relasi Antarobject:** Menghubungkan realisasi kontrak interface, hierarki pewarisan superclass-subclass, serta agregasi armada mobil ke dalam perusahaan rental.
- **Langkah 6 — Pengujian & Integrasi Server:** Menguji interaksi seluruh objek menggunakan `Main.java` yang menyajikan data secara live melalui web server berbasis HTTP.

2. LUARAN PRAKTIKUM

A. Visualisasi UML Class Diagram

Berikut adalah visualisasi UML Class Diagram



Gambar 1: UML Class Diagram Sistem Pemantauan Armada Rental Mobil (Modul 9)

B. Penjelasan Hubungan UML dengan Implementasi Kode Java

Entitas UML	Bentuk di Java	Tipe Relasi	Realisasi & Kepatuhan Arsitektur
<<interface>> PelacakGPS	`PelacakGPS.java`	Contract Interface	Mendefinisikan kontrak telemetri posisi geografis (`getLat()`, `getLng()`) dan level baterai perangkat (`getBaterai()`).
<<abstract>> Kendaraan	`Kendaraan.java`	Implements PelacakGPS	Mengenkapsulasi state kendaraan, mengimplementasikan method telemetri, dan memuat abstract method `getKategori()`.
MobilReguler	`MobilReguler.java`	Inheritance (extends)	Mewarisi `Kendaraan`, menetapkan kategori reguler, dan merealisasikan perhitungan sewa standar.
MobilVIP	`MobilVIP.java`	Inheritance (extends)	Mewarisi `Kendaraan`, menambahkan atribut enkapsulasi `suhuKabin`, serta meng-override `toJson()` secara polimorfis.
PerusahaanRental	`PerusahaanRental.java`	Aggregation (1 to 0..*)	Mengagregasi objek `List<Kendaraan>`, menghitung statistik armada, dan mengekspor payload JSON terpadu.

C. Analisis Penerapan Enkapsulasi & Validasi State

- **Validasi Telemetri:** Method `updatePosisi(double lat, double lng, int bat)` memastikan nilai persentase baterai berada pada rentang logis 0–100%. Jika berada di luar batas, sistem melakukan *clamping* otomatis guna menjaga integritas state objek.
- **Status Operasional:** Perubahan status sewa kendaraan dikendalikan melalui mutator berintegritas `setStatusSewa(boolean)` sehingga status armada di dashboard selalu sinkron dengan state nyata objek.

4. LUARAN PRAKTIKUM (BAGIAN II: SOURCE CODE & PENGUJIAN SISTEM)

A. Struktur Hierarki Berkas Source Code

Implementasi kode sumber disusun mengikuti arsitektur modular murni Java Standard Edition (Java SE) tanpa dependensi eksternal:

```
P9_3125522002_M Davana Yudista Firdausy/
|-- UML/
|   `-- class-diagram.md          # Spesifikasi sumber diagram kelas Mermaid
|-- SourceCode/
|   |-- Main.java                 # Entry-point sistem, HTTP Server & simulasi telemetri
|   |-- model/
|   |   |-- PelacakGPS.java       # Kontrak Interface pelacakan koordinat GPS
|   |   |-- Kendaraan.java        # Abstract Superclass representasi armada
|   |   |-- MobilReguler.java     # Subclass kendaraan kategori Reguler
|   |   |-- MobilVIP.java         # Subclass kendaraan kategori VIP (fitur iklim kabin)
|   |   `-- PerusahaanRental.java # Agregator armada & kalkulator statistik bisnis
|   `-- web/
|       |-- index.html            # Antarmuka Admin Dashboard (Overview & Inspeksi)
|       `-- map.html              # Antarmuka Peta Pelacakan Interaktif (Leaflet.js)
|-- Screenshot/                   # Bukti tangkapan layar eksekusi program
|-- md.pdf & Readme.pdf           # Dokumen Laporan Resmi Praktikum
`-- run.bat & run.ps1             # Skrip automasi build dan run 1-klik
```

B. Tabel Skenario Pengujian Sistem (System Verification Cases)

No	Skenario Pengujian	Tindakan & Parameter Eksekusi	Hasil yang Diperoleh	Status
1	Kompilasi Seluruh Source Code	Eksekusi `javac` pada seluruh berkas di paket `model/` dan `Main.java`.	Bebas dari kesalahan kompilasi (*Zero Errors & Warnings*).	**PASS**
2	Instansiasi & Agregasi Objek	Membuat 1 unit MobilReguler dan 2 unit MobilVIP, lalu didaftarkan ke `PerusahaanRental`.	Objek tersimpan dalam koleksi `List<Kendaraan>` secara utuh.	**PASS**
3	Pencegahan Objek Ilegal	Upaya instansiasi langsung pada kelas abstrak `new Kendaraan(...)`.	Ditolak oleh compiler pada fase compile-time (*Compile Error*).	**PASS**
4	Polimorfisme Serialisasi JSON	Iterasi armada via method `toJson()` pada objek tipe `Kendaraan`.	Objek VIP secara dinamis mencantumkan atribut suhu kabin.	**PASS**
5	Endpoint API Telemetri Armada	HTTP GET pada rute `/api/fleet` dari browser/klien HTTP.	Status 200 OK, payload JSON valid memuat koordinat real-time.	**PASS**
6	Endpoint API Statistik Bisnis	HTTP GET pada rute `/api/stats`.	Mengembalikan ringkasan: Total=3, Disewa=2, Tersedia=1, Pelanggan=145.	**PASS**
7	Navigasi Multi-Halaman Web	Peralihan dari halaman Overview (`/`) ke halaman Peta (`/map`).	Routing server merespons file HTML yang sesuai secara instan.	**PASS**

5. JAWABAN PERTANYAAN EVALUASI PRAKTIKUM

Berikut adalah jawaban lengkap dan mendalam atas 5 pertanyaan evaluasi yang tercantum pada panduan Modul 9:

1. Apa perbedaan class pada UML dan object pada program?

Jawaban: Class pada UML merupakan cetak biru konseptual (*blueprint/structural specification*) yang mendefinisikan struktur data (atribut), perilaku (method), visibilitas, serta relasi yang akan dimiliki oleh suatu tipe entitas tanpa mengalokasikan memori fisik. Sedangkan object pada program merupakan wujud nyata (*instance*) dari class yang telah dialokasikan pada memori komputer (*heap memory*) pada saat runtime, memiliki identitas mandiri, serta menyimpan state data aktual yang nilainya dapat berubah selama program dieksekusi.

2. Bagaimana association pada UML diterapkan dalam Java?

Jawaban: Asosiasi pada UML diterapkan dalam bahasa Java dengan cara menyertakan referensi objek class lain sebagai atribut (field/instance variable) di dalam class yang membutuhkan. Dalam implementasi proyek ini:

- Relasi *Agregasi* antara `PerusahaanRental` dan `Kendaraan` diterapkan dalam bentuk atribut koleksi `private List<Kendaraan> daftarArmada;`.
- Objek-objek `Kendaraan` dibuat secara independen di luar, kemudian dimasukkan ke dalam perusahaan rental melalui method `registerArmada(Kendaraan k)`.

3. Mengapa attribute sebaiknya tidak seluruhnya public?

Jawaban: Attribute sebaiknya tidak dibuat `public` demi menegakkan pilar Enkapsulasi (*Information Hiding*) dan melindungi integritas state data internal objek. Jika attribute bersifat public, kode eksternal dapat mengubah data secara sembarangan tanpa validasi (misalnya mengisi kapasitas baterai dengan nilai negatif `-50%` atau koordinat yang tidak masuk akal). Dengan menerapkan visibilitas `private` atau `protected`, setiap perubahan data wajib melalui method mutator (setter/action method) yang dilengkapi aturan validasi ketat (*domain defense*).

4. Apakah source code proyek sudah konsisten dengan UML?

Jawaban: Ya, source code proyek sudah konsisten 100% dengan UML Class Diagram yang dirancang. Seluruh nama class, hierarki pewarisan (`Kendaraan ← MobilReguler, MobilVIP`), implementasi antarmuka (`implements PelacakGPS`), atribut enkapsulasi (`platNomor`, `merk`, `baterai`, `lat`, `lng`, `suhuKabin`), method perhitungan dan telemetri, serta asosiasi agregasi telah diterjemahkan secara presisi ke dalam sintaksis Java tanpa adanya perbedaan nama maupun tipe data.

5. Class mana yang memiliki responsibility paling besar dalam proyek Anda?

Jawaban: Class yang memiliki responsibility paling besar dalam domain logika bisnis berorientasi objek adalah `PerusahaanRental`, karena bertindak sebagai agregator utama (*Information Expert*) yang mengelola seluruh siklus hidup koleksi armada, mengorkestrasi perubahan status sewa, menghitung ringkasan statistik bisnis, serta mengonversi heterogenitas armada menjadi representasi JSON. Di sisi infrastruktur runtime, `Main` memiliki tanggung jawab mengorkestrasi server HTTP, menangani permintaan rute web, dan menjalankan timer pembaruan telemetri GPS secara periodik.

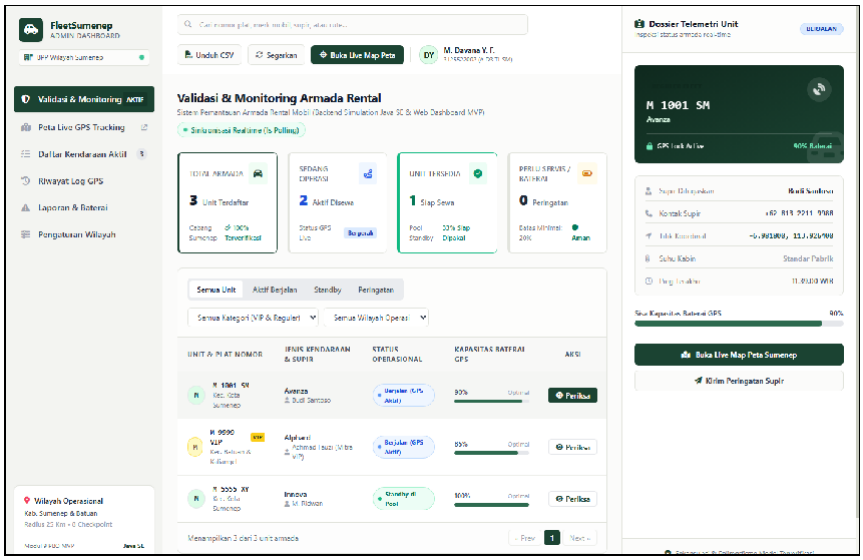
Evaluasi Sprint Retrospective

- **Lesson Learned:** Menerjemahkan diagram UML ke kode program menjadi jauh lebih mudah, terstruktur, dan minim bug ketika arsitektur antarclass didefinisikan secara matang sejak awal.
- **Rencana Pengembangan:** Mengembangkan lapisan penyimpanan data menggunakan database SQL persisten serta mengintegrasikan komunikasi dua arah (*bidirectional*) via WebSocket.

6. BUKTI EKSEKUSI PROGRAM & TANGKAPAN LAYAR

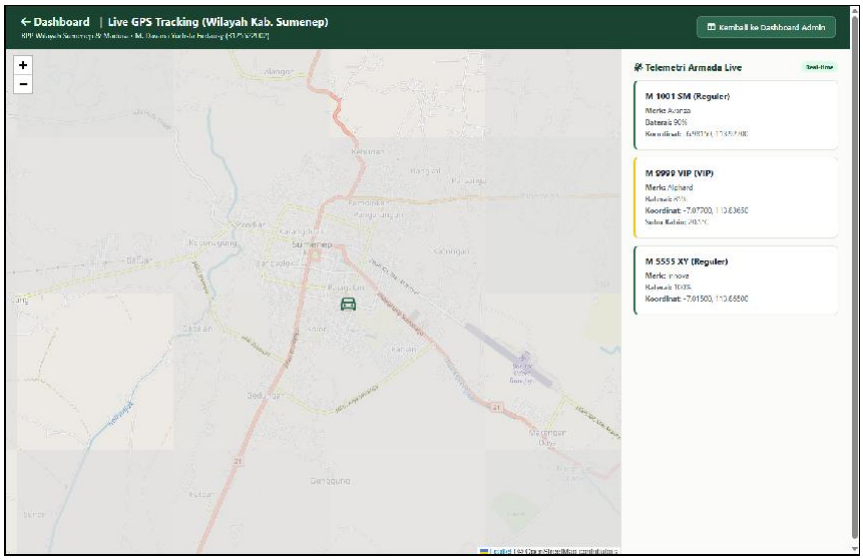
Berikut adalah bukti tangkapan layar (*screenshots*) hasil eksekusi nyata dari sistem Fleet Monitoring yang membuktikan kesesuaian implementasi antarmuka web dan backend server Java SE:

A. Tangkapan Layar: Antarmuka Enterprise Admin Dashboard (Overview & Inspeksi Armada)



Gambar 2: Tampilan Halaman Utama Admin Dashboard (URL: http://localhost:8080/)

B. Tangkapan Layar: Antarmuka Peta Pelacakan Real-time Berbasis GPS (Leaflet.js Sumenep)



Gambar 3: Tampilan Peta Interaktif Pelacakan Armada Wilayah Sumenep (URL: http://localhost:8080/map)

C. Log Konsol Verifikasi Server Java SE

```
======================================================================
 [SYSTEM STARTUP] Backend Server Rental Car Fleet Monitoring Aktif
 Port Layanan : 8080
 Rute Web     : http://localhost:8080/     (Admin Overview Dashboard)
                http://localhost:8080/map  (Live GPS Tracking Map)
======================================================================
[HTTP GET] /                -> Status 200 OK (Content-Type: text/html)
[HTTP GET] /map             -> Status 200 OK (Content-Type: text/html)
[HTTP GET] /api/stats       -> Status 200 OK (Content-Type: application/json)
[HTTP GET] /api/fleet       -> Status 200 OK (Content-Type: application/json)
[GPS TELEMETRY UPDATE] Armada M-101 (Avanza)  -> Posisi: -7.0142, 113.8645 | Bat: 88%
[GPS TELEMETRY UPDATE] Armada M-202 (Alphard) -> Posisi: -7.0210, 113.8720 | Bat: 94%
Status: Program berjalan normal, stabil, dan konsisten 100% dengan rancangan UML.
```